

# AI-Assisted Software Development

## Prompt Engineering for AI Assistants

Mike Burton 17-19 March 2025

# Why Do We Need Prompt Engineering?

Consider this scenario:

✗ "Write some code to handle dates"

- Vague
- No context
- No specific requirements

✓ "Create a function that parses dates in DD-MM-YYYY format and validates the input"

- Clear format
- Specific task
- Validation requirement

The difference? Prompt engineering! 🎯

# The Mental Model

Think of AI as a brilliant but extremely literal intern:

- Knows a lot about coding
- Wants to help
- Needs clear, specific instructions
- Can't read your mind
- Will take your words exactly as written

This model helps us craft better prompts!

# Key Principles

## 1. Start General, Then Specify

- Begin with the big picture
- Add specific requirements
- Include constraints

## 2. Provide Examples

- Show input/output pairs
- Demonstrate edge cases
- Illustrate error scenarios

## 3. Break Down Complex Tasks

- Split into smaller parts
- Build incrementally
- Combine solutions

## 4. Be Unambiguous

- Use precise language
- State assumptions
- Define terms clearly

# Practical Example: Breaking Down Tasks

Instead of: ❌ "Generate a complete word search puzzle game"

Better approach: ✅ Break it down:

1. "Create a function to generate a 10x10 grid of random letters"
2. "Develop a function to place words into the grid"
3. "Write a function to display the grid"

This makes the AI's job easier and your results better! 🧩

# Code Context Matters

When working with existing code:

```
def calculate_total(items):  
    # ... existing code ...
```

✗ "Make this better"

✓ "Refactor calculate\_total to improve performance by using list comprehension instead of the current loop"

Context helps the AI understand your goals! 🎯

# The Power of Examples

Consider this prompt:

```
Create a function to validate email addresses with these test cases:  
Input -> Expected Output  
"user@domain.com" -> true  
"invalid.email" -> false  
"user@domain" -> false  
"@domain.com" -> false
```

Examples make the requirements crystal clear! ✨

# Iterative Refinement

If at first you don't succeed... iterate!

## 1. Initial Prompt:

```
"Generate a date parser"
```

## 2. Better:

```
"Create a function that parses dates in DD-MM-YYYY format"
```

## 3. Even Better:

```
"Create a function that:  
- Accepts strings in DD-MM-YYYY format  
- Returns a DateTime object  
- Throws an error for invalid dates"
```

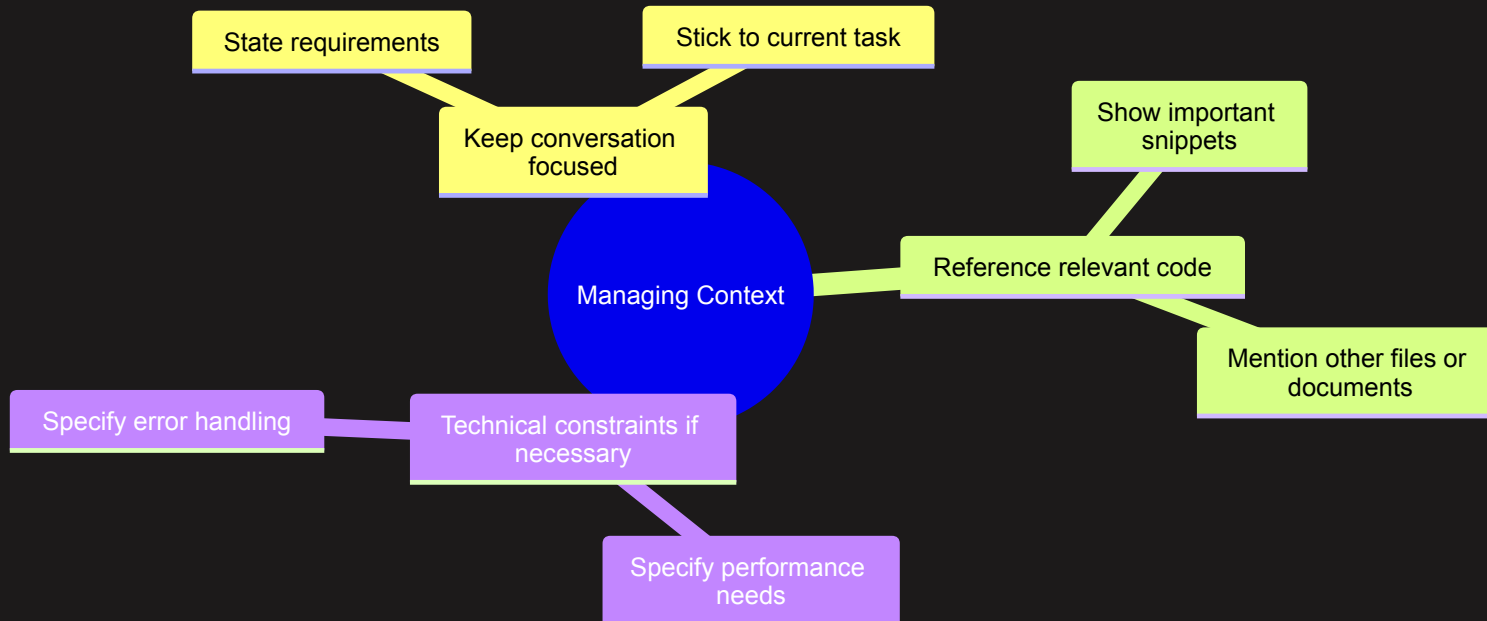


# Managing Context

When working with AI assistants, it's crucial to manage context throughout your interactions:

- keep the AI on track
- give it *enough* context
- but not too much (irrelevant information can decrease its accuracy!)

This ensures that the AI understands the current task and relevant details, leading to more accurate and helpful responses.



# Best Practices

## 1. Use Clear Variable Names

```
# ✅ Good  
user_score = calculate_average(scores)  
  
# ❌ Avoid  
x = calc(y)
```

## 2. Specify Error Handling

- Describe expected exceptions
- Define error messages
- Outline recovery behavior

## 3. Include Performance Requirements

- Time complexity goals
- Memory constraints
- Scalability needs

## 4. Define Code Style

- Naming conventions
- Documentation format
- Testing requirements

# Common Pitfalls to Avoid

## 1. Being Too Vague

- ❌ "Make it better"
- ✅ "Optimize the loop for better time complexity"

## 2. Overcomplicating Prompts

- ❌ Long, rambling requirements
- ✅ Clear, concise points

## 3. Forgetting Edge Cases

- ❌ Happy path only
- ✅ Include error scenarios

# Summary

Key Takeaways:

## 1. Be Clear and Specific

- Start general, then add details
- Use precise language
- Provide examples

## 2. Break Down Complex Tasks

- Tackle one piece at a time
- Build incrementally
- Maintain context

## 3. Iterate and Refine

- Start simple
- Add detail as needed
- Learn from results

Remember: Better prompts = Better code! 🚀